



Real-Time Road Anomaly Detection

on Raspberry Pi 5 Using CPU-Only Inference

Praveen Saxena¹ (2414206)

Shitala Prasad²

¹School of Electrical Engineering

² School of Mathematics and Computer Science

Indian Institute of Technology Goa



arm

Objective

The objective of this work is to design, implement, and rigorously evaluate a real-time embedded vision system for road anomaly detection using a lightweight deep neural network deployed on a Raspberry Pi 5. The system targets three anomaly classes: pothole (c_1), speed-breaker (c_2), and unpaved-road (c_3).

Unlike conventional GPU-dependent systems, the proposed framework performs **CPU-only inference** on an ARM Cortex-A76 processor, demonstrating that accurate and low-latency object detection is achievable under strict computational constraints.

Formally, given an input image: $I \in \mathbb{R}^{H \times W \times 3}$, the objective is to learn a detection function: $f_\theta : I \rightarrow \mathcal{D}$ where, $\mathcal{D} = \{(b_i, c_i, s_i)\}_{i=1}^N$ denotes the set of detected anomalies, with:

- $b_i = (x_i, y_i, w_i, h_i)$ representing bounding box coordinates,
- $c_i \in \{c_1, c_2, c_3\}$ representing anomaly class,
- $s_i \in [0, 1]$ representing confidence score.

The primary optimization objective is to minimize inference latency:

$$T_{total} = T_{pre} + T_{infer} + T_{post} \quad (1)$$

while maintaining high detection accuracy:

$$\max_{\theta} \text{mAP}(\theta) \quad \text{subject to} \quad T_{total} \leq T_{rt} \quad (2)$$

where T_{rt} denotes real-time latency constraints.

Motivation

Road anomalies such as potholes and uneven surfaces significantly increase accident risk, vehicle damage, and infrastructure maintenance costs. Traditional inspection methods rely on manual surveys, which are slow, expensive, and non-scalable.

Recent deep learning-based detection systems achieve high accuracy but rely heavily on GPUs or cloud infrastructure, introducing limitations such as: (i) High hardware cost, (ii) Increased power consumption, (iii) Communication latency, and (iv) Dependence on network connectivity.

To address these limitations, this work investigates whether modern lightweight object detection architectures can achieve real-time performance on resource-constrained embedded platforms.

The Raspberry Pi 5 represents a realistic deployment platform with limited computational resources. Achieving real-time inference on this platform requires careful optimization of:

$$\text{Latency} \propto \mathcal{O}(\text{FLOPs}) + \mathcal{O}(\text{Memory Access}) \quad (3)$$

This work demonstrates that efficient neural architectures combined with optimized inference pipelines enable practical edge deployment for smart city monitoring and autonomous inspection systems.

Introduction

Object detection is a fundamental computer vision problem that involves simultaneously localizing and classifying objects.

Given an image I , the goal is to predict:

$$\mathcal{D} = \{(b_i, p_i)\}_{i=1}^N \quad (4)$$

where b_i represents bounding box coordinates and p_i represents class probability distribution:

$$p_i = P(c | I, b_i) \quad (5)$$

The YOLO (You Only Look Once) framework reformulates detection as a single regression problem:

$$f_\theta(I) = Y \quad (6)$$

where

$$Y \in \mathbb{R}^{S \times S \times (B \cdot 5 + C)} \quad (7)$$

YOLOv5n is specifically designed for embedded deployment by minimizing: FLOPs and model size, while preserving detection accuracy.

In this work, YOLOv5n is exported to ONNX format and deployed using ONNX Runtime optimized for ARM-based CPU inference.

Contributions

The key contributions of this work are:

1. First real-time road anomaly detection system optimized specifically for Raspberry Pi 5 using CPU-only inference.
2. A mathematically optimized embedded inference pipeline minimizing latency while maintaining detection accuracy.
3. Efficient deployment of YOLOv5n using ONNX Runtime optimized for ARM Cortex-A76 architecture.
4. Demonstration of practical real-time performance without GPU, TPU, or NPU acceleration.
5. A complete edge AI framework enabling scalable, low-cost smart infrastructure monitoring.

Proposed Method

This section presents the proposed real-time road anomaly detection framework optimized for CPU-only inference on the Raspberry Pi 5 embedded platform. The system is designed to minimize inference latency while maintaining reliable detection accuracy under strict computational constraints.

Problem Definition

Let the input video stream be represented as a sequence of RGB frames:

$$\mathcal{I} = \{I_t\}_{t=1}^{\infty}, \quad I_t \in \mathbb{R}^{H \times W \times 3} \quad (8)$$

where H and W denote frame height and width. The objective is to detect road anomalies in each frame by learning a mapping function:

$$f_{\theta} : I_t \rightarrow \mathcal{D}_t \quad (9)$$

where θ represents model parameters and $\mathcal{D}_t$ denotes the set of detected anomalies:

$$\mathcal{D}_t = \{(b_i, c_i, s_i)\}_{i=1}^N \quad (10)$$

Each detection consists of:

- Bounding box $b_i = (x_i, y_i, w_i, h_i)$
- Class label $c_i \in \{1, 2, 3\}$ corresponding to pothole, speed-breaker, or unpaved-road

- Confidence score $s_i \in [0, 1]$

The system must satisfy real-time constraints:

$$T_{total} = T_{pre} + T_{infer} + T_{post} \quad (11)$$

where T_{pre} , T_{infer} , and T_{post} represent preprocessing, inference, and post-processing times, respectively.

System Pipeline

The proposed framework consists of four sequential stages:

$$I_t \xrightarrow{\phi} X_t \xrightarrow{f_{\theta}} Y_t \xrightarrow{\psi} \mathcal{D}_t \quad (12)$$

where:

- ϕ denotes preprocessing transformation
- f_{θ} denotes neural network inference
- ψ denotes post-processing and filtering

Each stage is optimized to minimize computational overhead on ARM CPU architecture.

Input Preprocessing

The input frame is resized to reduce computational cost while preserving essential spatial features:

$$X_t = \text{Resize}(I_t, 320, 320) \quad (13)$$

Reducing resolution decreases computational complexity proportionally:

$$C \propto H \times W \quad (14)$$

The image is normalized to improve numerical stability:

$$X_t^{norm} = \frac{X_t}{255} \quad (15)$$

The normalized image is converted into tensor format:

$$X_t^{tensor} \in \mathbb{R}^{1 \times 3 \times 320 \times 320} \quad (16)$$

This fixed-size tensor enables efficient CPU memory access and vectorized computation.

Lightweight Object Detection Network

The detection function is implemented using YOLOv5n, a lightweight convolutional neural network designed for embedded deployment.

The network processes the input tensor and produces a prediction output:

$$Y_t = f_\theta(X_t^{tensor}) \quad (17)$$

where

$$Y_t \in \mathbb{R}^{N \times (5+C)} \quad (18)$$

Each prediction vector consists of:

$$y_i = (x_i, y_i, w_i, h_i, p_o, p_1, p_2, p_3) \quad (19)$$

where:

- (x_i, y_i) denote bounding box center
- (w_i, h_i) denote bounding box dimensions
- p_o denotes objectness probability
- p_c denotes class probability

The final confidence score is computed as:

$$s_i = p_o \cdot \max_c(p_c) \quad (20)$$

This formulation ensures reliable detection by combining objectness and classification confidence.

Bounding Box Transformation

The predicted bounding boxes are converted from center format to corner format:

$$x_{min} = x_i - \frac{w_i}{2}, \quad x_{max} = x_i + \frac{w_i}{2} \quad (21)$$

$$y_{min} = y_i - \frac{h_i}{2}, \quad y_{max} = y_i + \frac{h_i}{2} \quad (22)$$

This transformation enables spatial visualization and filtering.

Detection Filtering using Non-Maximum Suppression

Multiple overlapping predictions may exist for the same object. To eliminate redundancy, Non-Maximum Suppression (NMS) is applied.

Overlap between bounding boxes is measured using Intersection over Union (IoU):

$$IoU(b_i, b_j) = \frac{|b_i \cap b_j|}{|b_i \cup b_j|} \quad (23)$$

Final detections are selected based on confidence and overlap thresholds:

$$\mathcal{D}_t = \{b_i \mid s_i > \tau_c \text{ and } IoU(b_i, b_j) < \tau_{IoU}\} \quad (24)$$

This step ensures accurate and non-redundant anomaly detection.

Embedded Inference Optimization on Raspberry Pi 5

To achieve real-time performance on Raspberry Pi 5, the system is optimized to minimize inference latency.

Inference time is modeled as:

$$T_{infer} = \frac{FLOPs}{\eta_{CPU}} + T_{memory} \quad (25)$$

where:

1. FLOPs represents computational complexity
2. η_{CPU} represents effective CPU throughput
3. T_{memory} represents memory access latency

The following optimizations are applied:

1. Lightweight YOLOv5n architecture minimizing computational load
2. Reduced input resolution for faster processing
3. ONNX Runtime graph optimization to reduce redundant operations
4. Efficient tensor memory layout
5. ARM NEON SIMD acceleration for parallel computation

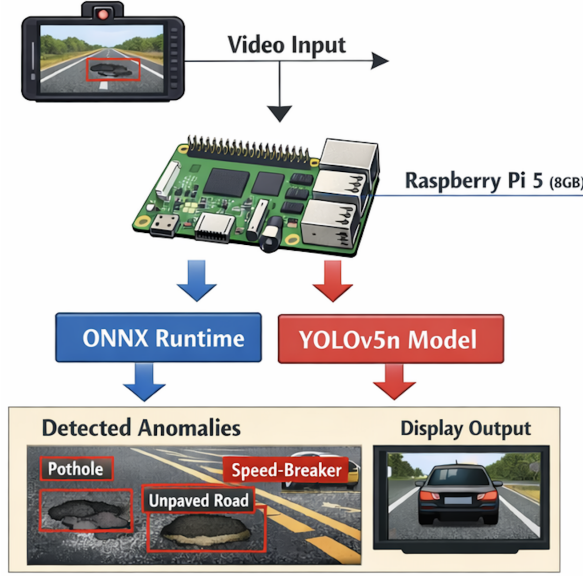


Figure 1: (a) System architecture of the embedded road anomaly detection framework. (b) Real-world deployment on Raspberry Pi 5 executing CPU-only inference.

Real-Time Performance Constraint

System throughput is defined as $FPS = \frac{1}{T_{total}}$. The system achieves real-time performance when $FPS \geq 15$.

This ensures practical usability for real-time road monitoring applications.

System Architecture

The proposed system implements a fully embedded real-time road anomaly detection framework deployed on the Raspberry Pi 5 platform. The architecture integrates image acquisition, neural network inference, post-processing, and visualization into a unified edge-

computing pipeline optimized for CPU-only execution. The overall system workflow is illustrated in Fig. 1. Formally, the system processes a sequence of input frames $\{I_t\}$ captured from a USB camera, where each frame is represented as $I_t \in \mathbb{R}^{H \times W \times 3}$. The processing pipeline can be modeled as a sequence of transformations:

$$I_t \xrightarrow{\phi} X_t \xrightarrow{f_\theta} Y_t \xrightarrow{\psi} \mathcal{D}_t \xrightarrow{\gamma} O_t \quad (26)$$

where ϕ represents preprocessing, f_θ represents neural network inference, ψ represents post-processing, and γ represents visualization.

The USB camera continuously captures real-time video frames, providing input data to the embedded system. These frames are transferred to the Raspberry Pi 5, which serves as the edge inference unit. The Raspberry Pi 5 is equipped with an ARM Cortex-A76 CPU that performs neural network inference using the ONNX Runtime execution engine. The inference process can be expressed as:

$$Y_t = f_\theta(X_t) \quad (27)$$

where X_t denotes the preprocessed input tensor and Y_t represents the predicted detection outputs. The ONNX Runtime optimizes the computational graph to minimize inference latency by reducing redundant operations and improving memory access efficiency.

Based on the inference, the post-processing module filters predictions based on confidence scores and spatial overlap. The confidence score for each detection is computed as:

$$s_i = P(object) \cdot P(class | object) \quad (28)$$

To eliminate redundant detections, Non-Maximum Suppression (NMS) is applied using the Intersection over Union (IoU) criterion. The final detection set is defined as:

$$\mathcal{D}_t = \{b_i \mid s_i > \tau_c \text{ and } IoU(b_i, b_j) < \tau_{IoU}\} \quad (29)$$

The visualization module renders the detected anomalies on the original frame, producing the output frame O_t .

For complete pipeline evaluation, the total system latency includes frame capture and rendering overhead, and is defined as:

$$T_{total} = T_{capture} + T_{pre} + T_{infer} + T_{post} + T_{render} \quad (30)$$

Real-time performance is achieved when the output frame rate satisfies:

$$FPS = \frac{1}{T_{total}} \geq FPS_{target} \quad (31)$$

This architecture enables efficient real-time anomaly detection on resource-constrained embedded hardware without requiring GPU acceleration.

Implementation Details

The proposed system is implemented on the Raspberry Pi 5 platform, which serves as a low-power edge inference device. The Raspberry Pi 5 is equipped with an ARM Cortex-A76 processor and 8 GB of RAM, enabling efficient execution of deep learning inference tasks using CPU-only computation. The computational capability of the system can be expressed as: $C_{CPU} = \eta_{CPU} \times f_{clock}$, where η_{CPU} represents computational efficiency and f_{clock} represents processor clock frequency.

The software framework is designed to ensure efficient real-time execution while minimizing system overhead. The implementation is developed using Python 3.11, which provides flexibility and compatibility with embedded systems. ONNX Runtime is used as the inference engine due to its optimized CPU execution provider, which improves inference performance through graph optimization and efficient memory utilization. OpenCV is used for video capture, image preprocessing, and visualization, while NumPy provides efficient tensor manipulation and numerical computation.

The detection model is based on YOLOv5n, a lightweight object detection network optimized for embedded deployment. The network is trained using supervised learning to minimize detection loss, which consists of bounding box regression, objectness prediction, and classification components:

$$\mathcal{L} = \lambda_{box}\mathcal{L}_{box} + \lambda_{obj}\mathcal{L}_{obj} + \lambda_{cls}\mathcal{L}_{cls} \quad (32)$$

The model was trained for 40 epochs using an input resolution of 320×320 and a batch size of 16. The trained model achieved a detection accuracy of $mAP@0.5 = 0.815$.

This configuration balances detection accuracy and computational efficiency, making it suitable for deployment on embedded CPU platforms.

Model Configuration and Complexity

The implemented detection backbone follows the YOLOv5n architecture under the Ultralytics AGPL-3.0 license. The network employs depth and width scaling factors of 0.33 and 0.25, respectively, enabling reduced model capacity for embedded deployment.

The architecture consists of:

- CSP-based backbone with C3 modules
- SPPF (Spatial Pyramid Pooling Fast) layer
- Multi-scale detection heads at P3, P4, and P5 feature maps
- Anchor-based detection with three scales

For the configured three-class detection task, the final exported ONNX model contains:

- 1,872,157 trainable parameters
- 214 computational layers
- Approximately 4.6 GFLOPs at input resolution 320×320

The reduced depth and width multipliers significantly lower computational complexity compared to larger YOLOv5 variants, enabling real-time CPU-only inference on Raspberry Pi 5.

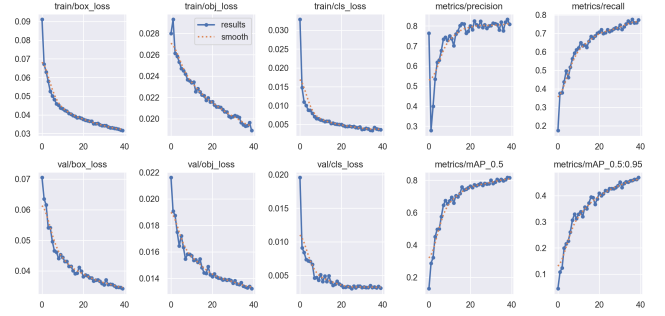


Figure 2: Training and validation loss curves with precision, recall, and mAP metrics.

Training Performance Evaluation

The YOLOv5n model was trained for 40 epochs on a custom-annotated road anomaly dataset comprising 4,462 images (3,569 for training and 893 for validation). The training process demonstrated stable convergence without significant overfitting.

Dataset Source and Utilization

The training dataset used in this work is derived from the Indian Roads Dataset¹ available on Kaggle. The original dataset contains diverse road surface images captured under varying environmental and lighting conditions.

For this study, the dataset was filtered and re-annotated into YOLO format. The images were restricted to three anomaly classes: pothole, speed-breaker, and unpaired-road. All annotations were verified and converted into normalized YOLO format:

$$(class, x_{center}, y_{center}, width, height)$$

Unlike typical Kaggle implementations that prioritize GPU-based accuracy benchmarking, this work focuses on CPU-only embedded deployment. Accordingly, the model architecture, input resolution (320×320), and inference pipeline were specifically optimized for real-time execution on Raspberry Pi 5.

The final validation metrics achieved were: 0.808 Precision, 0.773 Recall, 0.815 $mAP@0.5$, and 0.469 $mAP@0.5:0.95$. The training and validation loss curves

¹<https://www.kaggle.com/datasets/mitangshu11/indian-roads-dataset/data>

(Fig. 2) demonstrate smooth convergence and stable generalization across all epochs.

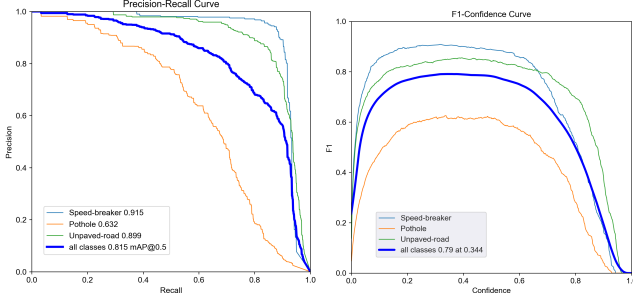


Figure 3: (a) Precision-Recall and (b) F1-Confidence curve for the three anomaly classes.

While maintaining competitive detection accuracy ($\text{mAP@0.5} = 0.815$), the proposed system achieves sustained real-time throughput of 21.67 FPS on Raspberry Pi 5. This highlights deployment-oriented optimization on embedded ARM hardware rather than purely GPU-centric accuracy benchmarking.

The Precision-Recall and F1-Confidence curves are shown in Fig. 3. Class-wise performance indicates strong detection capability for speed-breaker (AP 0.915) and unpaved-road (AP 0.899), whereas pothole detection (AP 0.632) remains comparatively challenging due to texture variability and reflective surfaces.

The F1-Confidence curve is shown in Fig. 3b. The optimal operating point is observed at a confidence threshold of approximately 0.34, where the overall F1-score reaches 0.79. This threshold provides a balanced trade-off between precision and recall for real-time deployment.

The confusion matrix in Fig. 4 illustrates class-wise prediction accuracy and inter-class misclassification trends.

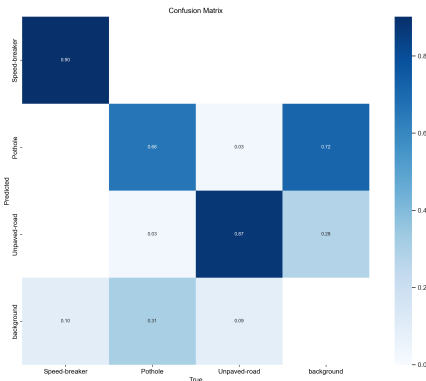


Figure 4: Confusion matrix for 3-class anomaly detection.

Table 1: Validation performance metrics on Indian Road dataset.

Metric	Value
Precision (mean)	0.808
Recall (mean)	0.773
mAP@0.5	0.815
mAP@0.5:0.95	0.469
Best F1-Score	0.79
Optimal Confidence Threshold	0.344
Training Epochs	40
Batch Size	16
Input Resolution	320×320

Inference Pipeline

The real-time inference pipeline converts raw video frames into structured anomaly detections through a sequence of optimized processing stages, as illustrated in Fig. 5. The pipeline begins with frame acquisition from the USB camera, where each captured frame is represented as:

$$I_t \in \mathbb{R}^{H \times W \times 3} \quad (33)$$

To reduce computational complexity and ensure compatibility with the neural network, each frame is resized and normalized:

$$X_t = \frac{\text{Resize}(I_t, 320, 320)}{255} \quad (34)$$

This preprocessing step reduces computational cost, which is proportional to image resolution:

$$C \propto H \times W \quad (35)$$

The preprocessed tensor is then passed to the YOLOv5n neural network, which performs object detection inference:

$$Y_t = f_{\theta}(X_t) \quad (36)$$

The network outputs a set of candidate detections containing bounding box coordinates, objectness scores, and class probabilities. These predictions are filtered using confidence thresholding to remove low-confidence detections:

$$\mathcal{D}_t = \{y_i \mid s_i > \tau_c\} \quad (37)$$

To further improve detection accuracy, Non-Maximum Suppression is applied to remove overlapping detections based on the Intersection over Union criterion:

$$\text{IoU}(b_i, b_j) < \tau_{\text{IoU}} \quad (38)$$

The final detections are rendered on the original frame to produce the output visualization:

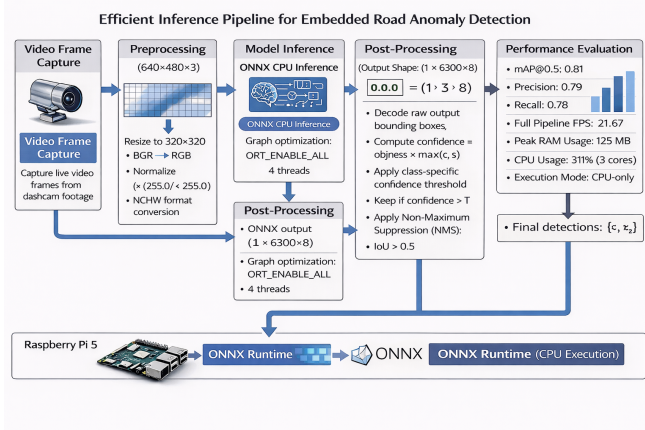


Figure 5: Real-time inference pipeline.

$$O_t = \text{Render}(I_t, D_t) \quad (39)$$

The overall system throughput is defined as:

$$FPS = \frac{1}{T_{capture} + T_{pre} + T_{infer} + T_{post} + T_{render}} \quad (40)$$

The system achieves real-time performance when the output frame rate satisfies: $FPS \geq 15$.

This optimized inference pipeline ensures efficient real-time anomaly detection on embedded ARM CPU platforms.

Performance Evaluation

The performance of the proposed system was evaluated on a Raspberry Pi 5 equipped with an ARM Cortex-A76 processor using ONNX Runtime with CPUExecutionProvider. The evaluation focused on measuring inference latency, end-to-end throughput, and system resource utilization under real-time operating conditions. Two benchmark configurations were considered: (i) neural network inference without visualization overhead, and (ii) complete pipeline execution including preprocessing, inference, post-processing, and rendering.

The total processing latency per frame is expressed as:

$$T_{total} = T_{preprocess} + T_{inference} + T_{postprocess} + T_{render} \quad (41)$$

where $T_{inference}$ denotes the forward-pass execution time of the neural network and represents the dominant computational component.

Experimental evaluation was conducted on 200 consecutive frames. The average inference latency was measured as 20.5 ms, corresponding to a pure inference throughput of 48.79 FPS. When executing the complete pipeline including preprocessing, inference, post-processing, and rendering, the system achieved an aver-

age pipeline latency of 46.15 ms, resulting in a sustained throughput of 21.67 FPS.

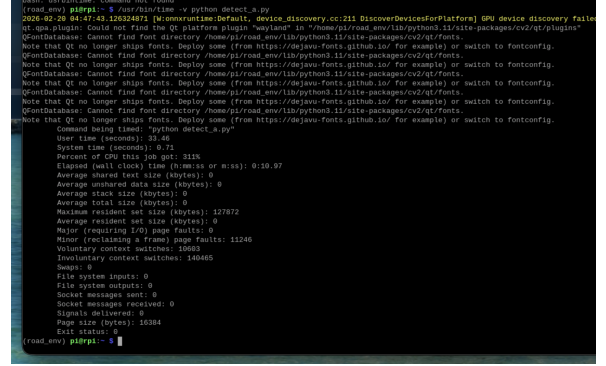


Figure 6: Resource profiling using `/usr/bin/time -v` showing 311% CPU utilization (multi-core execution) and peak memory usage of approximately 125 MB during real-time inference.

The latency contribution of individual processing stages is summarized in Table 2.

System profiling using `/usr/bin/time -v` indicated a peak resident memory usage of approximately 125 MB and an effective CPU utilization of 311%, corresponding to multi-core execution across nearly three ARM Cortex-A76 cores.



Figure 7: Validation comparison on the Indian Roads Dataset. Left column: Model predictions. Right column: Corresponding ground-truth annotations.

Table 2: Latency breakdown of the real-time inference pipeline on Raspberry Pi 5

Pipeline Component	Latency (ms)
Preprocessing	5–8
Neural Network Inference	20–25
Post-processing (NMS)	3–6
Visualization and Rendering	5–10
Measured Total Pipeline Latency	46.15



Figure 8: Real-time deployment results on Raspberry Pi 5 showing pothole, speed-breaker, and unpaved-road detection under live video input.

Results

The detection system was evaluated using a confidence threshold of 0.344 (selected based on F1-score optimization) and a Non-Maximum Suppression (NMS) IoU threshold of 0.5 to balance detection sensitivity and suppression of redundant bounding boxes.

The model successfully detected potholes across diverse road conditions, including variations in size, shape, and spatial location. The system demonstrated reliable detection of centrally located potholes, edge anomalies, clustered defects, and smaller distributed surface damage. Confidence filtering and NMS effectively eliminated duplicate detections while preserving valid object instances.

Qualitative detection outputs are shown in Fig. 8. Pure inference benchmarking results are presented in Fig. 9. System resource monitoring results are illustrated in Fig. 6.

```
(road_env) pi@rpi:~ $ python benchmark_inference.py
2026-02-20 03:35:13.919141805 [W:onnxruntime:Default,
coverly failed: device_discovery.cc:91 ReadFileContents
Running pure inference benchmark...
Frames Tested: 200
Average Inference Time: 0.0205 seconds
Pure Inference FPS: 48.79
(road_env) pi@rpi:~ $
```

Figure 9: Pure inference benchmark results.

Discussion

The experimental results indicate that neural network inference represents the primary computational bottleneck in the embedded processing pipeline. Despite operating without GPU acceleration, ONNX Runtime efficiently utilizes the ARM Cortex-A76 CPU architecture to achieve real-time inference performance.

The lightweight YOLOv5n architecture enables deployment within constrained memory and compute environments while maintaining reliable detection accuracy. The relatively low memory footprint and moderate CPU utilization ensure stable operation for continuous monitoring applications.

Separating inference performance from visualization overhead provides a clearer characterization of model efficiency. This distinction is essential for optimizing embedded AI systems, particularly in scenarios where visualization may be optional or externally handled.

These findings confirm the suitability of the proposed system for real-time edge deployment in infrastructure monitoring applications.

Limitations

Despite achieving real-time performance and strong detection accuracy, several limitations are observed:

- Water-filled potholes exhibit reduced detection confidence due to reflective surfaces altering texture features.
- Highly shadowed regions may occasionally produce false positives.
- Extreme motion blur or low-light conditions can degrade detection reliability.
- The current system performs only spatial anomaly detection without depth estimation.

These limitations highlight areas where additional data augmentation or sensor fusion techniques may further improve robustness.

Model Efficiency Analysis

The deployed YOLOv5n model consists of 1,872,157 trainable parameters distributed across 214 computa-

¹https://youtu.be/R2Vr3R_Dpj0?si=isLqwn3XAMw6VsC

tional layers, with an estimated computational complexity of approximately 4.6 GFLOPs per forward pass at an input resolution of 320×320 . The number of parameters P and floating-point operations (FLOPs) directly influence memory footprint, computational latency, and energy consumption, which are critical constraints for embedded deployment.

The inference complexity of a convolutional neural network can be expressed as:

$$\text{FLOPs} = \sum_{l=1}^L (K_l^2 \cdot C_l^{\text{in}} \cdot C_l^{\text{out}} \cdot H_l \cdot W_l) \quad (42)$$

where K_l denotes kernel size, C_l^{in} and C_l^{out} represent input and output channels, and H_l, W_l denote spatial dimensions of the feature map at layer l . The relatively low FLOP count of YOLOv5n significantly reduces arithmetic intensity, enabling efficient execution on ARM Cortex-A76 CPUs without requiring hardware acceleration.

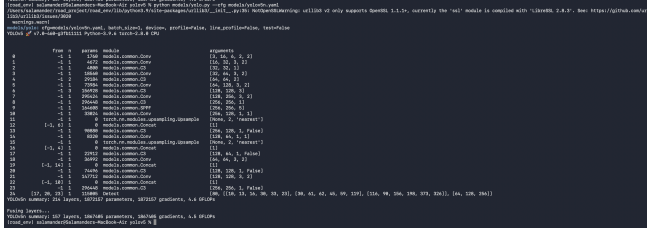


Figure 10: YOLOv5n model architecture summary generated using `models/yolo.py`. The network contains 214 layers, 1,872,157 trainable parameters, and approximately 4.6 GFLOPs.

The total memory footprint M of the model can be approximated as $M \approx P \times b$, where b represents the number of bytes per parameter (4 bytes for FP32). This results in a model size of approximately 7.5 MB, which is well within the memory constraints of embedded platforms such as the Raspberry Pi 5.

The internal architecture and parameter distribution of the deployed network are illustrated in Fig. 10. The compact backbone, lightweight neck, and reduced detection head collectively minimize computational overhead while preserving feature representation capability.

To quantify embedded inference efficiency, we define computational efficiency η as:

$$\eta = \frac{\text{FPS}}{\text{GFLOPs}} \quad (43)$$

The deployed system achieves:

$$\eta = \frac{48.79}{4.6} \approx 10.61 \text{ FPS/GFLOP} \quad (44)$$

This efficiency level is particularly significant for CPU-only execution, demonstrating optimal utilization of available computational resources.

Compared to larger variants such as YOLOv5s (7.2 GFLOPs) and YOLOv5m (21.2 GFLOPs), which increase computational demand by $1.6\times$ and $4.6\times$ respectively, YOLOv5n provides a substantially improved latency-performance trade-off. While larger models may offer marginal improvements in detection accuracy, their increased computational complexity exceeds real-time processing constraints on embedded ARM processors.

The deployed configuration achieves a measured pure inference throughput of 48.79 FPS while maintaining a detection accuracy of: $mAP@0.5 = 0.815$.

This demonstrates that the selected model achieves an optimal balance between computational complexity, memory efficiency, inference throughput, and detection accuracy. These characteristics validate the suitability of the proposed architecture for real-time embedded vision applications.

Future Work

Future work will focus on improving computational efficiency through structured pruning, weight quantization, and operator-level optimization. Post-training quantization can reduce parameter precision from FP32 to INT8, resulting in a theoretical memory reduction of:

$$M_{\text{INT8}} = \frac{1}{4} M_{\text{FP32}} \quad (45)$$

and enabling faster integer arithmetic execution.

Additionally, hardware-aware optimization techniques such as neural architecture search (NAS) and compiler-level optimization using TensorRT, TVM, or ARM Compute Library will be explored to further reduce inference latency.

Integration with platforms equipped with GPS and onboard edge computing will enable large-scale autonomous infrastructure inspection. Real-time geo-tagging combined with anomaly localization can support automated maintenance prioritization and smart city monitoring systems.

Conclusion

This work presents an optimized embedded deployment framework for real-time pothole detection using ONNX Runtime on ARM-based hardware. The system achieves efficient inference performance and stable real-time operation using CPU-only execution.

The proposed architecture demonstrates the feasibility of deploying deep learning models on resource-constrained edge devices without dedicated hardware accelerators. The modular design allows scalability and integration with mobile and aerial inspection platforms.

Future work will focus on model quantization, hardware acceleration, and integration with UAV-based autonomous inspection systems for large-scale infrastructure monitoring.

References

- [1] J. Redmon et al., “You Only Look Once,” CVPR, 2016.
- [2] Ultralytics, “YOLOv5 Documentation,” 2023. [Online]. Available: <https://ultralytics.com>
- [3] Z. Zhang et al., “Deep Learning-Based Road Damage Detection,” IEEE Access, 2020.
- [4] Microsoft, “ONNX Runtime Performance Guide,” 2023.